

JavaScript var, let, const

[Back to JS page](#)

Table of Contents

- [JavaScript var, let, const](#)
 - [Scope Rules](#)
 - [Example 1](#)
 - [Reassignment vs. Redeclaration](#)
 - [Example 2](#)
 - [Hoisting and the Temporal Dead Zone \(TDZ\)](#)
 - [Example 3](#)
 - [Best Practice](#)
 - [Example 4](#)
 - [Document](#)
 - [Reference](#)

Difference Between var, let and const

The table below summarizes the differences:

	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function	Block	Block
Redeclaration	Yes	No	No
Reassignment	Yes	Yes	No
Hoisted	Yes	No (TDZ)	No (TDZ)

Scope Rules

`var` is function-scoped, while `let` and `const` are block-scoped:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript var, let, const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Scope Rules</h4>
<p id="demo"></p>

<script>
{
  var varX = 1;      // Function scoped (accessible outside block)
  let letX = 2;      // Block scoped
  const constX = 3;  // Block scoped
}

document.getElementById("demo").innerHTML =
  "var (function scope): " + varX + " - accessible<br>" +
  "let (block scope): not accessible outside<br>" +
  "const (block scope): not accessible outside";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Reassignment vs. Redeclaration

`var` can be redeclared and reassigned. `let` can be reassigned but NOT redeclared. `const` can be neither:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript var, let, const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reassignment vs. Redeclaration</h4>
<p id="demo"></p>

<script>
// var - can be redeclared and reassigned
var name = "John";
var name = "Jane";      // Redeclaration OK
name = "Bob";           // Reassignment OK

// let - can be reassigned but NOT redeclared
let age = 30;
age = 31;                // Reassignment OK
// let age = 32;         // Redeclaration ERROR

// const - cannot be reassigned or redeclared
const city = "New York";
// city = "London";     // Reassignment ERROR
// const city = "LA";   // Redeclaration ERROR

document.getElementById("demo").innerHTML =
  "var name: " + name + " (redeclared + reassigned)<br>" +
  "let age: " + age + " (reassigned)<br>" +
  "const city: " + city + " (cannot change)";
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Hoisting and the Temporal Dead Zone (TDZ)

`var` is hoisted and initialized with `undefined`. `let` and `const` are hoisted but NOT initialized (Temporal Dead Zone):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript var, let, const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hoisting and the Temporal Dead Zone (TDZ)</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// var is hoisted and initialized with undefined
document.getElementById("demo1").innerHTML =
  "var x: " + x + " (hoisted, initialized as undefined)";
var x = 5;

// let is NOT initialized before declaration (TDZ)
try {
  console.log(y); // Would throw ReferenceError
  let y = 10;
} catch(err) {
  document.getElementById("demo2").innerHTML =
    "let y: Error - " + err.message + "<br>" +
    "let is in Temporal Dead Zone before declaration";
}
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Best Practice

The best practice is to always use `const` unless you know the value will change. Use `let` if you plan to reassign. Avoid `var` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript var, let, const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Best Practice</h4>
<p id="demo"></p>

<script>
// Best practice: use const by default
const PI = 3.14159;
const MAX_USERS = 100;

// Use let when reassignment is needed
let counter = 0;
counter = counter + 1;
counter++;

// Avoid var in modern JavaScript
// var oldStyle = "not recommended";

let text = "Best practices:<br><br>";
text += "1. Use <b>const</b> by default:<br>";
text += "    const PI = " + PI + "<br>";
text += "    const MAX_USERS = " + MAX_USERS + "<br><br>";
text += "2. Use <b>let</b> when reassigning:<br>";
text += "    let counter = " + counter + "<br><br>";
text += "3. Avoid <b>var</b> in modern code";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript var, let, const](#)